Design and Implementation of a 5-Stage Pipelined RISC-V Processor with 2-Bit Branch Predictor

From Sequential Datapath to Pipelined Architecture
with Data Forwarding, Hazard Detection and Branch Prediction

Team Name: Avengers
Team Number: 19

A. Harsha Vardhan (2024102040)
S. V. Santhosh (2024102054)
S. Sangameshwar (2024102017)

Abstract

This report presents the complete design, implementation, and verification of a 64-bit RISC-V processor in Verilog. We began with a working sequential (single-cycle) datapath that could execute basic RISC-V instructions one at a time. Building on that foundation, we upgraded the design into a fully pipelined 5-stage processor with four pipeline registers, a data forwarding unit, a hazard detection unit, and a 2-bit saturating counter branch predictor. The processor supports R-type arithmetic and logical instructions (ADD, SUB, AND, OR, XOR), I-type immediate operations (ADDI), load/store (LD/SD), and conditional branching (BEQ). We verified the pipelined design with a 15-instruction test program that exercises forwarding paths, memory operations, and branch prediction, confirming that the pipeline produces correct results while overlapping instruction execution for improved throughput.

Contents

1 Introduction **4**

2 Instruction Set Architecture **4**
2.1 Supported Instructions 5
2.2 Instruction Encoding Formats 5

3 Sequential Processor (The Starting Point) **5**
3.1 How the Sequential Design Worked 5
3.2 Why We Moved to Pipelining 6

4 Pipelined Processor Architecture **6**
4.1 The Five Pipeline Stages 6
4.1.1 Stage 1: Instruction Fetch (IF) 6
4.1.2 Stage 2: Instruction Decode (ID) 7
4.1.3 Stage 3: Execute (EX) 7
4.1.4 Stage 4: Memory Access (MEM) 7
4.1.5 Stage 5: Write Back (WB) 8
4.2 Pipeline Registers 8

5 The ALU in Detail **8**
5.1 ALU Sub-Modules 9

6 Control Unit **9**

7 Hazard Handling **9**
7.1 Data Hazards and Forwarding 9
7.1.1 How the Forwarding Unit Works 10
7.2 Load-Use Hazard and Stalling 10
7.3 Control Hazards (Branch Handling) 11
7.4 Branch Predictor 12
7.4.1 Branch History Table (BHT) 12
7.4.2 Prediction (IF Stage) 12
7.4.3 Update (ID Stage) 12
7.4.4 Misprediction Recovery 13

8 Register File Design **13**

9 Immediate Generator **13**

10 Memory Subsystem **14**
10.1 Instruction Memory 14
10.2 Data Memory 14

11 Top-Level Processor Module **14**
11.1 PC Update Logic 15
11.2 Halt Mechanism 15
11.3 Wrapper Module: Sequential to Pipelined Transition 16

11.4 Testbench (`pipe_tb.v`) 16

12 Test Program and Verification **17**
12.1 The Test Program 17
12.2 Why This Program Is a Good Test 17
12.3 Expected Register State After Execution 18

13 GTKWAVE output **19**

14 Pipeline Execution Trace **19**

15 Branch Execution in Detail **20**

16 Module Hierarchy and File Organization **21**

17 What Changed: Sequential to Pipelined **21**
17.1 Key Design Decisions 22

18 Simulation and Results **23**
18.1 Simulation Setup 23
18.2 Results Summary 23

19 Challenges and Lessons Learned **24**

20 Conclusion **24**

1 Introduction

When we first started working on this project, the goal was straightforward: build a processor that can understand and execute RISC-V instructions. We began with the simplest approach possible—a sequential processor where each instruction completes fully before the next one starts. It was easy to reason about, easy to debug, and it gave us confidence that our individual components (ALU, register file, memory, control unit) were all working correctly.

But in real-world processors, executing one instruction at a time is painfully slow. The ALU just sits idle while the processor is busy fetching the next instruction from memory. The register file does nothing while the ALU is crunching numbers. All that hardware, and most of it is wasting time doing nothing each cycle.

That is exactly why pipelining exists. The idea is beautifully simple: break the work of executing an instruction into stages, and let different instructions occupy different stages at the same time. While one instruction is being decoded, the next one is already being fetched. While one instruction is using the ALU, the previous one is accessing memory. It is like an assembly line in a factory—no single worker does the entire job, but the factory produces one finished product every cycle.

Of course, pipelining introduces its own headaches. What happens when one instruction needs a result that the previous instruction has not finished computing yet? What if a branch instruction changes where we should be fetching from? These are real problems, and solving them is a big part of what makes this project interesting.

In this report, we walk through the entire journey—from the sequential design to the pipelined version—and explain how every component works, how we handle hazards, and how we verified that everything produces the correct output.

2 Instruction Set Architecture

Our processor implements a subset of the RV64I base integer instruction set. We kept the scope focused on the most fundamental operations that demonstrate all the key concepts of pipelining.

2.1 Supported Instructions

Table 1: Supported RISC-V Instructions

Type	Instruction	Opcode	Description
R-type	`ADD`	`0110011`	Addition
R-type	`SUB`	`0110011`	Subtraction
R-type	`AND`	`0110011`	Bitwise AND
R-type	`OR`	`0110011`	Bitwise OR
R-type	`XOR`	`0110011`	Bitwise XOR
I-type	`ADDI`	`0010011`	Add Immediate
I-type	`LD`	`0000011`	Load Doubleword
S-type	`SD`	`0100011`	Store Doubleword
B-type	`BEQ`	`1100011`	Branch if Equal

2.2 Instruction Encoding Formats

Every RISC-V instruction is exactly 32 bits wide. The beauty of the RISC-V encoding is that the opcode always sits in bits [6:0], and register fields are always in the same positions regardless of the instruction type. This makes decoding much simpler.

- **R-type:** Used for register-to-register operations. Fields: `funct7[31:25]`, `rs2[24:20]`, `rs1[19:15]`, `funct3[14:12]`, `rd[11:7]`, `opcode[6:0]`.
- **I-type:** Used for immediate arithmetic and loads. Fields: `imm[31:20]`, `rs1[19:15]`, `funct3[14:12]`, `rd[11:7]`, `opcode[6:0]`.
- **S-type:** Used for stores. The immediate is split: `imm[31:25]` and `imm[11:7]`.
- **B-type:** Used for branches. The immediate is split and shifted for 2-byte alignment.

3 Sequential Processor (The Starting Point)

Before we could build the pipeline, we needed all the fundamental building blocks to be solid. The sequential processor was our testbed—a design where one instruction flows through the entire datapath in a single clock cycle.

3.1 How the Sequential Design Worked

In the sequential version, every clock cycle looked like this:

1. The **Program Counter (PC)** holds the address of the current instruction.
2. The **Instruction Memory** reads the 32-bit instruction at that address.
3. The **Control Unit** decodes the opcode and generates control signals.
4. The **Register File** reads the source registers (rs1 and rs2).

5. The **Immediate Generator** sign-extends the immediate field to 64 bits.
6. The **ALU** performs the required operation.
7. The **Data Memory** handles loads and stores.
8. The result is **written back** to the register file.
9. The **PC updates** to PC+4 (or a branch target).

All of this happened in one cycle. The clock period had to be long enough to accommodate the slowest path through the entire datapath—from instruction memory read all the way through the ALU and data memory back to the register file. That is the critical path, and it determines the maximum clock frequency.

3.2 Why We Moved to Pipelining

The problem with the sequential design is utilization. At any given moment, most of the hardware is idle. The ALU is doing nothing while the instruction memory is being read. The data memory sits idle while the ALU works. Pipelining solves this by keeping every part of the processor busy every cycle.

4 Pipelined Processor Architecture

The pipelined processor breaks execution into five classic stages. Between each pair of stages, we inserted a **pipeline register** that captures and holds all the signals that the next stage will need. This way, five different instructions can be "in flight" simultaneously, each in a different stage.

4.1 The Five Pipeline Stages

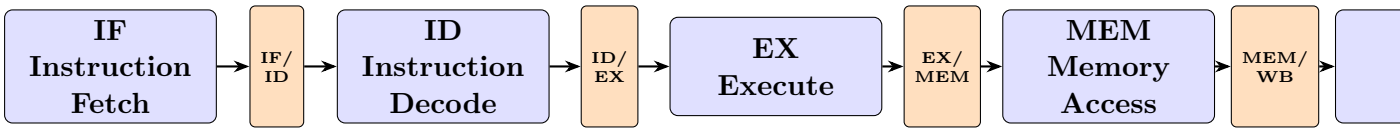


Figure 1: The five pipeline stages separated by four pipeline registers.

4.1.1 Stage 1: Instruction Fetch (IF)

This is the front door of the processor. The PC register holds the current instruction address, and the instruction memory returns the 32-bit instruction sitting at that address. Meanwhile, a dedicated adder computes PC+4 in parallel so we already know where the next instruction lives. The PC multiplexer decides whether to go to PC+4 (normal sequential flow), hold the current PC (stall), or jump to a branch target.

Modules involved:

- `pc` — 64-bit program counter register with synchronous reset
- `pc_add` — Ripple-carry adder that computes PC + 4
- `instruction_mem` — 4096-byte ROM that loads from `instructions.txt`

4.1.2 Stage 2: Instruction Decode (ID)

Once the instruction arrives through the IF/ID pipeline register, the decode stage figures out what to do with it. The control unit looks at the opcode and sets flags like "this is a memory read," "write the result back to a register," "this is a branch," and so on. The register file reads out the values of the two source registers (rs1 and rs2). The immediate generator extracts the immediate value from the instruction and sign-extends it to 64 bits.

One important design decision we made: **branch resolution happens in the ID stage itself**. Rather than waiting until the EX stage (which would mean two wasted cycles on a taken branch), we compare rs1 and rs2 right here using a simple equality check. If BEQ detects equality, the branch is taken immediately, the IF/ID register is flushed, and we start fetching from the branch target. This gives us only a 1-cycle branch penalty.

Modules involved:

- `control_unit` — Generates all control signals from opcode
- `register_file` — 32 × 64-bit register file with internal forwarding
- `imm_gen` — Immediate generator supporting I, S, and B formats
- `hazard_detection_unit` — Detects load-use hazards and triggers stalls

4.1.3 Stage 3: Execute (EX)

This is where the heavy lifting happens. The ALU performs the actual computation—addition, subtraction, logic operations, shifts, comparisons—whatever the instruction asks for. But before the ALU can work, the forwarding multiplexers check whether either operand needs to come from a later pipeline stage (more on this in Section 7.1). The ALU control unit decodes `funct3` and `funct7` along with the ALU operation type to figure out exactly which operation the ALU should perform.

Modules involved:

- `alu_64_bit` — 64-bit ALU supporting ADD, SUB, AND, OR, XOR
- `alu_control` — Translates aluop + funct3 + funct7 into ALU opcode
- `forwarding_unit` — Generates forwarding control signals
- `mux2` — ALU source B multiplexer (register vs. immediate)

4.1.4 Stage 4: Memory Access (MEM)

Only load and store instructions actually do anything interesting here. For a store (`SD`), the data from the second source register is written to the data memory at the address computed by the ALU. For a load (`LD`), we read from that address. For all other instructions, this stage just passes the ALU result through.

Modules involved:

- `data_mem` — 1024-byte data memory with little-endian byte ordering

4.1.5 Stage 5: Write Back (WB)

The final stage writes the result back to the destination register. A multiplexer selects between the ALU result (for arithmetic/logic instructions) and the data memory output (for loads). The register file write enable ensures we only write when the instruction actually produces a result.

Modules involved:

- `mux2` — Selects between ALU result and memory data
- `register_file` — Write port activated by WB stage

4.2 Pipeline Registers

The pipeline registers are the glue that makes the whole pipeline work. They are simple D-type flip-flop arrays that capture signals on the rising clock edge and hold them steady for the next stage.

Table 2: Pipeline Register Contents

Register	Signals Carried
IF/ID	PC (64-bit), Instruction (32-bit), Predicted Taken (1-bit), Predict Target (64-bit). Supports `stall` (hold current values) and `flush` (clear to NOP on misprediction).
ID/EX	PC, rd1, rd2, immediate (all 64-bit); rs1, rs2, rd (5-bit each); control signals (alusrc, aluop, funct3, funct7_5, branch, memread, memwrite, memtoreg, regwrite). Flushed on stall to insert a bubble.
EX/MEM	ALU result, rd2 (store data), rd (all 64/5-bit); zero flag; control signals (branch, memread, memwrite, memtoreg, regwrite).
MEM/WB	ALU result, memory data read, rd; control signals (memtoreg, regwrite).

5 The ALU in Detail

The ALU is the computational heart of the processor. Rather than using behavioral Verilog shortcuts, we built each operation from structural sub-modules and then multiplexed their outputs based on the ALU opcode.

5.1 ALU Sub-Modules

Table 3: ALU Operations and Their Opcodes

Opcode	Operation	Implementation Module
0000	ADD	`adder64` — 64-bit ripple-carry adder
0100	XOR	`xor_gate` — Bitwise XOR with zero detection
0110	OR	`or_gate` — Bitwise OR with zero detection
0111	AND	`and_gate` — Bitwise AND with zero detection
1000	SUB	`sub64` — Subtraction via 2's complement

All sub-modules run in parallel. The ALU instantiates every operation simultaneously and then uses a large multiplexer to select the correct result based on the opcode. This is a classic "compute everything, pick one" approach. It keeps the design simple and makes the critical path predictable.

6 Control Unit

The control unit is a purely combinational block that looks at the 7-bit opcode and generates all the control signals the datapath needs. We kept it lean—just the essential signals.

Table 4: Control Signal Generation by Instruction Type

Instruction	RegWrite	ALUSrc	MemRead	MemWrite	MemToReg	Branch	ALUO
R-type	1	0	0	0	0	0	10
ADDI	1	1	0	0	0	0	00
LD	1	1	1	0	1	0	00
SD	0	1	0	1	0	0	00
BEQ	0	0	0	0	0	1	01

7 Hazard Handling

Pipelining would be easy if every instruction were independent. In reality, instructions depend on each other all the time, creating hazards. Our processor handles two major types of hazards: data hazards and control hazards.

7.1 Data Hazards and Forwarding

A data hazard occurs when an instruction needs a value that a previous instruction has not yet written to the register file. Consider this sequence:

```
addi x1, x0, 7      # writes x1
add  x3, x1, x2     # reads x1 -- but x1 isn't in the register file yet!
```

Without forwarding, `add` would read a stale value of `x1`. The solution is the **forwarding unit**, which detects when a result is available in a later pipeline stage and routes it directly to the ALU inputs, bypassing the register file entirely.

7.1.1 How the Forwarding Unit Works

The forwarding unit compares the source registers of the instruction currently in the EX stage (`ex_rs1`, `ex_rs2`) against the destination registers of instructions in the MEM and WB stages:

- **Forward from MEM** (`forwardA/B = 10`): If the instruction in the MEM stage writes to a register that the EX instruction reads, forward the ALU result from EX/MEM.
- **Forward from WB** (`forwardA/B = 01`): If the instruction in the WB stage writes to a register that the EX instruction reads (and there is no MEM forward), forward the write-back value.
- **No forward** (`forwardA/B = 00`): Use the value from the register file.

The forwarding unit also checks that the destination register is not `x0` (since `x0` is hardwired to zero and should never be forwarded).

```
always @(*) begin
    // ForwardA
    if (mem_regwrite && (mem_rd != 0) && (mem_rd == ex_rs1))
        forwardA = 2'b10;  // Forward from EX/MEM
    else if (wb_regwrite && (wb_rd != 0) && (wb_rd == ex_rs1))
        forwardA = 2'b01;  // Forward from MEM/WB
    else
        forwardA = 2'b00;  // No hazard

    // ForwardB (same logic for second operand)
    if (mem_regwrite && (mem_rd != 0) && (mem_rd == ex_rs2))
        forwardB = 2'b10;
    else if (wb_regwrite && (wb_rd != 0) && (wb_rd == ex_rs2))
        forwardB = 2'b01;
    else
        forwardB = 2'b00;
end
```

Listing 1: Forwarding Unit Logic

7.2 Load-Use Hazard and Stalling

There is one situation that forwarding alone cannot solve: when a load instruction is immediately followed by an instruction that needs the loaded value.

```
ld   x9, 0(x0)      # x9 won't be available until the end of MEM stage
add  x12, x9, x3    # needs x9 in EX stage -- too early!
```

The loaded value is not available until the end of the MEM stage, but the dependent instruction needs it at the beginning of the EX stage—that is one cycle too early even for forwarding. The **hazard detection unit** catches this and inserts a 1-cycle stall (bubble).

```
always @(*) begin
    if (ex_memread &&
        ((ex_rd == id_rs1) || (ex_rd == id_rs2)) &&
        (ex_rd != 5'd0))
        stall = 1'b1;
    else
        stall = 1'b0;
end
```

Listing 2: Hazard Detection Unit

When a stall is triggered:

1. The **IF/ID register holds** its current values (the dependent instruction stays in ID).
2. The **PC holds** its current value (no new fetch).
3. The **ID/EX register is flushed** (a NOP bubble enters the EX stage).

After the stall cycle, the load value is available in MEM/WB, and normal forwarding takes over.

7.3 Control Hazards (Branch Handling)

When a branch instruction is encountered, the pipeline has already fetched the next instruction speculatively. If the branch is taken, that fetched instruction is wrong and must be discarded.

Our design resolves branches in the **ID stage** rather than the EX stage. We compare rs1 and rs2 directly in ID using a simple equality check. To ensure the comparator sees the most recent values (even if the producing instruction has not yet written back), we added **branch comparator forwarding** from both the EX and MEM stages:

```
// Forward from EX (ALU result, not loads) and MEM
wire [63:0] branch_rs1_val =
    (ex_regwrite && !ex_memread && ex_rd != 0 && ex_rd == rs1)
        ? alu_result :
    (mem_regwrite && mem_rd != 0 && mem_rd == rs1)
        ? mem_fwd_val : rd1;

assign id_zero     = (branch_rs1_val == branch_rs2_val);
assign branch_taken = branch & (funct3 == 3'b000) & id_zero;
```

Listing 3: Branch Comparator with Forwarding

When the branch is taken:

1. The branch target is computed as: $\mathrm{PC}_{\mathrm{beq}} + (\mathrm{imm} \ll 1)$.
2. The IF/ID register is **flushed** (the incorrectly fetched instruction is killed).
3. The PC is updated to the branch target.

This gives us only a **1-cycle branch penalty** (instead of 2 cycles if we waited for EX stage resolution). A load-use hazard involving a branch source register is handled by the hazard detection unit, which stalls for one cycle so that the loaded value becomes available via MEM-stage forwarding.

7.4 Branch Predictor

To further reduce the branch penalty, we added a **2-bit saturating counter branch predictor** with a 16-entry Branch History Table (BHT). The predictor operates in the IF stage (making its prediction available before the instruction even enters ID), and is updated in the ID stage once the actual branch outcome is known.

7.4.1 Branch History Table (BHT)

The BHT contains 16 entries, each holding a 2-bit saturating counter. The table is indexed by bits [5:2] of the PC, giving us a simple and fast lookup.

Each counter can be in one of four states:

Table 5: 2-Bit Saturating Counter States

Value	State	Prediction
00	Strongly Not Taken	Not Taken
01	Weakly Not Taken	Not Taken
10	Weakly Taken	Taken
11	Strongly Taken	Taken

The prediction is simply the most significant bit of the counter: if `bht[index][1] == 1`, predict taken; otherwise, predict not taken. All counters are initialised to `00` (Strongly Not Taken) on reset.

7.4.2 Prediction (IF Stage)

During the IF stage, the predictor checks whether the fetched instruction is a BEQ (opcode `1100011`, funct3 `000`). If it is and the counter's MSB is 1, the predictor asserts `predict_taken` and supplies the predicted target address computed as $\text{PC} + (\text{imm} \ll 1)$:

```
assign predict_taken  = if_is_beq & bht[if_pc[5:2]][1];
assign predict_target = if_pc + if_imm_shifted;
```

Listing 4: Prediction Logic

The prediction and target are carried through the IF/ID pipeline register alongside the instruction and PC, so the ID stage knows what was predicted.

7.4.3 Update (ID Stage)

Once the branch resolves in the ID stage, the BHT entry is updated with a saturating increment (if taken) or decrement (if not taken):

```
always @(posedge clk) begin
    if (id_is_branch) begin
        if (id_actual_taken) begin
            if (bht[id_index] < 2'b11)
                bht[id_index] <= bht[id_index] + 1;
        end else begin
            if (bht[id_index] > 2'b00)
                bht[id_index] <= bht[id_index] - 1;
        end
```

```
    end
end
```

Listing 5: BHT Update Logic

7.4.4 Misprediction Recovery

A misprediction is detected when the predicted outcome differs from the actual outcome:

```
assign mispredict = id_is_branch & (id_predicted_taken != branch_taken)
    ;
assign flush_ifid = mispredict & !stall;
```

On a misprediction the IF/ID register is flushed (suppressed during a stall to avoid killing a held instruction), and the PC is corrected:

- If the branch was actually **taken** (but predicted not taken), the PC is set to the branch target.
- If the branch was actually **not taken** (but predicted taken), the PC is set to `id_pc + 4` to undo the speculative fetch.

8 Register File Design

The register file contains 32 registers, each 64 bits wide, following the RISC-V specification. It features:

- **Two read ports** (combinational) for rs1 and rs2.
- **One write port** (synchronous on posedge clk) for the destination register.
- **x0 hardwired to zero**: Any write to x0 is ignored, and reads from x0 always return 0.
- **Internal forwarding**: If the WB stage is writing to the same register that the ID stage is reading in the same cycle, the write data is forwarded directly to the read output. This avoids the "write-then-read in the same cycle" timing race.
- **Register dump**: Every negative clock edge, the entire register file contents are written to `register-file.txt` for debugging and verification.

9 Immediate Generator

The immediate generator extracts and sign-extends the immediate field from the 32-bit instruction to a full 64-bit value. It handles three instruction formats:

Table 6: Immediate Extraction by Format

Format	Instructions	Immediate Bits
I-type	ADDI, LD	`imm = {{52{instr[31]}}, instr[31:20]}`
S-type	SD	`imm = {{52{instr[31]}}, instr[31:25], instr[11:7]}`
B-type	BEQ	`imm = {{52{instr[31]}}, instr[31], instr[7], instr[30:25], instr[11:8]}`

The upper 52 bits are always filled with copies of `instr[31]` (the sign bit), ensuring correct sign extension for both positive and negative immediates.

10 Memory Subsystem

10.1 Instruction Memory

The instruction memory is a read-only byte-addressed memory that holds the program. It is initialized at simulation start by reading from `instructions.txt`. Each instruction is 32 bits (4 bytes), and the memory concatenates four consecutive bytes to form the instruction word:

$$\text{instr} = \{\text{mem}[\text{addr}],\ \text{mem}[\text{addr}+1],\ \text{mem}[\text{addr}+2],\ \text{mem}[\text{addr}+3]\}$$

Total capacity is 4096 bytes, allowing up to 1024 instructions.

10.2 Data Memory

The data memory is a read-write byte-addressed memory for load/store operations. It holds 1024 bytes and uses **little-endian** byte ordering:

- **Store (SD)**: The 64-bit value is split into 8 bytes. The least significant byte goes to the lowest address.
- **Load (LD)**: Eight consecutive bytes are read and concatenated in reverse order to reconstruct the 64-bit value.
- Writes are **synchronous** (on posedge clk), reads are **combinational** (immediate).

11 Top-Level Processor Module

The top-level module (`seq_processor`) ties everything together. It instantiates all five stages, the four pipeline registers, the forwarding unit, the hazard detection unit, and the branch predictor. Here is the big picture of how signals flow:

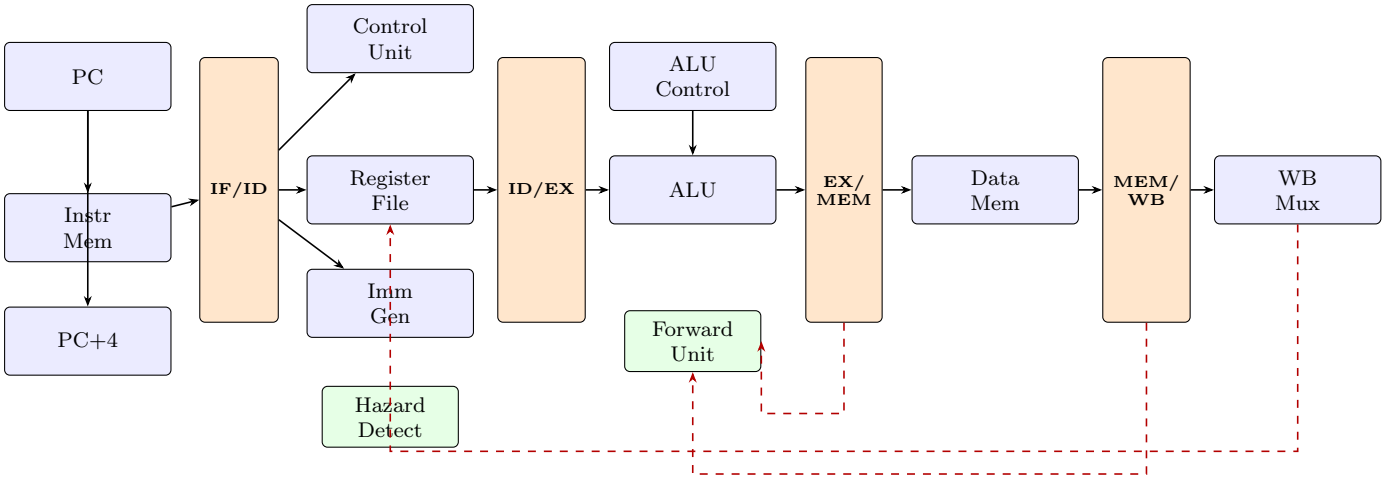


Figure 2: Simplified datapath of the 5-stage pipelined RISC-V processor. Dashed red arrows show forwarding and write-back paths.

11.1 PC Update Logic

The next PC value is determined by a priority-based selection that integrates the branch predictor:

```
assign next_pc = halt           ? pc_out        :  // Halt: freeze
                 stall          ? pc_out        :  // Stall: hold
                 mispredict     ? (branch_taken   // Mispredict:
                     correct
                      ? branch_addr : id_pc_plus4) :
                 bp_predict_taken ? bp_predict_target : // Predicted
                     taken
                                  pc4;              // Normal: PC + 4
```

The priority order is:

1. **Halt** — freeze the PC when the processor is shutting down.
2. **Stall** — hold the PC during a load-use hazard.
3. **Misprediction** — correct the PC when the branch predictor was wrong. If the branch was actually taken, jump to `branch_addr`; if actually not taken, go to `id_pc + 4`.
4. **Predicted taken** — speculatively follow the predictor's target.
5. **Default** — normal sequential flow with PC + 4.

11.2 Halt Mechanism

The processor uses a simple halt mechanism. When the instruction fetched at the current PC is all zeros (`0x00000000`), a `halt` flag is set on the next clock edge. Once `halt` is high, the PC freezes and the simulation stops — **no further writes** to the register file or data memory occur.

Importantly, the processor does **not** automatically drain the pipeline. Any instructions still in the ID, EX, MEM, or WB stages at the moment halt is detected will **not**

complete. It is the programmer's responsibility to place explicit NOP instructions (`add x0, x0, x0` = `0x00000033`) after the last real instruction so that all real instructions progress through all five stages before the halt sentinel (`0x00000000`) is reached in IF. Four NOPs are needed to ensure the last real instruction completes WB.

11.3 Wrapper Module: Sequential to Pipelined Transition

The top-level module `seq_processor.v` evolved significantly in the transition from sequential to pipelined design:

- **Sequential version**: The module contained a simple linear datapath — PC → instruction memory → control unit → register file → ALU → data memory → write-back — all in one combinational path within a single clock cycle.
- **Pipelined version**: The same components are now separated by four pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB). Additional modules were added:
  - `forwarding_unit` — detects data hazards and generates forwarding mux selects.
  - `hazard_detection_unit` — detects load-use hazards and triggers stalls.
  - `branch_predictor` — provides branch predictions in IF and updates BHT in ID.
  - Branch comparator forwarding logic — routes fresh values from EX/MEM stages to the ID-stage branch comparator.
  - Misprediction detection and recovery logic — detects when the predictor was wrong and corrects the PC.
  - Halt logic — freezes the processor when all-zeros instruction is fetched.

11.4 Testbench (`pipe_tb.v`)

The testbench module `pipe_tb` instantiates `seq_processor` as the unit under test and drives the simulation:

- **Clock generation**: A 10 ns clock (5 ns high, 5 ns low) is generated using `always #5 clk = ~clk`.
- **Reset**: The reset signal is held high for 20 ns (two full clock cycles) to initialise all pipeline registers, the PC, and the BHT, then released.
- **Cycle counter**: An integer `cycle_count` increments every rising clock edge after reset and stops when `halt` goes high.
- **Monitoring**: Every cycle, the testbench displays the current PC, fetched instruction, decoded instruction, register reads, ALU result, write-back signals, and branch predictor status (predicted/actual/misprediction).
- **Termination**: When `halt` goes high inside the processor, the testbench prints a summary and calls `$finish`.
- **Include files**: The testbench uses Verilog `` `include `` directives to pull in all submodules, making it the single compilation entry point.

12 Test Program and Verification

We verified the processor with a carefully designed 15-instruction test program that exercises every major feature: arithmetic, logic, shifts, memory operations, forwarding, and branching.

12.1 The Test Program

Table 7: Test Program Instructions

#	Assembly	Operation	Expected Result
I1	`addi x2, x0, 5`	$x2 = 0 + 5$	$x2 = 5$
I2	`addi x3, x0, 10`	$x3 = 0 + 10$	$x3 = 10$
I3	`add x1, x2, x3`	$x1 = x2 + x3 = 5 + 10$	$x1 = 15$
I4	`sub x2, x2, x3`	$x2 = x2 - x3 = 5 - 10$	$x2 = -5$
I5	`and x4, x3, x3`	$x4 = x3 \,\&\, x3$	$x4 = 10$
I6	`and x5, x3, x4`	$x5 = x3 \,\&\, x4 = 10 \,\&\, 10$	$x5 = 10$
I7	`or x6, x2, x4`	$x6 = x2 \mid x4 = (-5) \mid 10$	$x6 = -5$
I8	`or x7, x2, x3`	$x7 = x2 \mid x3 = (-5) \mid 10$	$x7 = -5$
I9	`sd x1, 0(x5)`	$\text{mem}[10] = 15$	Store 15
I10	`ld x10, 0(x5)`	$x10 = \text{mem}[10]$	$x10 = 15$
I11	`sd x6, 24(x5)`	$\text{mem}[34] = -5$	Store -5
I12	`ld x11, 24(x5)`	$x11 = \text{mem}[34]$	$x11 = -5$
I13	`beq x4, x5, +8`	$10 == 10 \rightarrow$ taken	Skip I14
I14	`beq x0, x0, 0`	*SKIPPED (flushed)*	—
I15	`add x13, x1, x10`	$x13 = 15 + 15$	$x13 = 30$
I16–I19: Four NOP instructions (`add x0, x0, x0`) for pipeline draining.			

12.2 Why This Program Is a Good Test

This test program was designed to cover key corner cases:

1. **Data forwarding from MEM stage**: When `add x1, x2, x3` (I3) executes, x2 was just written by I1 (now in MEM). The forwarding unit routes the value directly to the ALU.

2. **Data forwarding from WB stage**: When `sub x2, x2, x3` (I4) executes, x3 is forwarded from the WB stage since I2 has reached that point.

3. **Negative number handling**: I4 produces $5 - 10 = -5$ (two's complement `0xFFFFFFFFFFFFFFFB`). The OR operations (I7, I8) then combine this negative value with positive operands, testing that logical operations work correctly with signed values.

4. **Store/Load with register base**: I9 stores 15 to `mem[x5]` = `mem[10]`, and I10 loads it back into x10. Similarly, I11 stores −5 to `mem[x5+24]` = `mem[34]`, and I12 loads it back into x11. This verifies proper base+offset addressing.

5. **Branch taken with predictor**: I13 (`beq x4, x5, +8`) compares x4=10 with x5=10. They are equal, so the branch is taken. The branch predictor initially predicts not-taken (counter starts at `00`), causing a misprediction and a 1-cycle penalty.

6. **Branch flush**: I14 (`beq x0, x0, 0`) is fetched speculatively after I13 but gets flushed when the misprediction is detected.

7. **Core ALU operations**: ADD, SUB, AND, and OR are all exercised and their results verified against expected values.

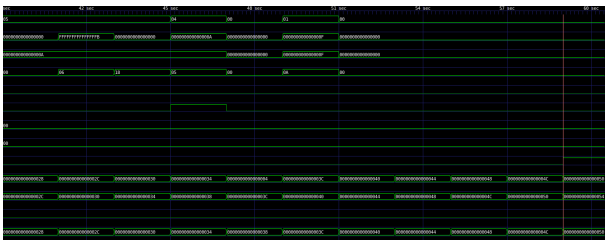
12.3 Expected Register State After Execution

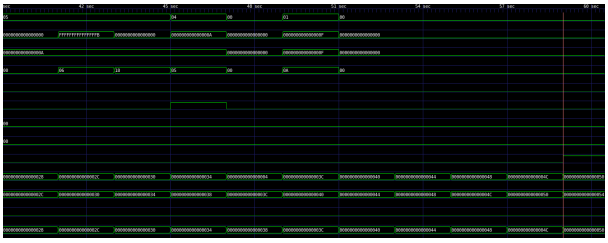
Table 8: Final Register File Contents (from `register_file.txt`)

Register	Value (Hex)	Explanation
x0	`0000000000000000`	Hardwired zero
x1	`000000000000000f`	15 = 5 + 10 (ADD)
x2	`fffffffffffffffb`	−5 = 5 − 10 (SUB, two's complement)
x3	`000000000000000a`	10 (from ADDI)
x4	`000000000000000a`	10 = 10 & 10 (AND)
x5	`000000000000000a`	10 = 10 & 10 (AND)
x6	`fffffffffffffffb`	−5 = (−5) | 10 (OR)
x7	`fffffffffffffffb`	−5 = (−5) | 10 (OR)
x8–x9	`0000000000000000`	Never written
x10	`000000000000000f`	15 (loaded from mem[10])
x11	`fffffffffffffffb`	−5 (loaded from mem[34])
x12	`0000000000000000`	Never written
x13	`000000000000001e`	30 = 15 + 15 (ADD)
x14–x31	`0000000000000000`	Never written

The instruction counter in `register_file.txt` reads **20**, which accounts for the 15 real program instructions, 4 pipeline-draining NOPs, and 1 flushed instruction slot from the branch misprediction.

13 GTKWAVE output





Explanation

In the source registers, the values are as expected given.
Halt is only on at the last cycle as can be seen in the graph.
There is no stall in this instruction set so stall is always 0.
At one branch cycle it flushes as it should be taken and it is as expected in the graph.
PC is increasing 4 for each cycle shown in the graph.

14 Pipeline Execution Trace

To really understand how the pipeline works, let us walk through the first few cycles in detail.

Table 9: Pipeline Execution Timeline (First 8 Cycles)

Cycle	IF	ID	EX	MEM	WB
1	I1	—	—	—	—
2	I2	I1	—	—	—
3	I3	I2	I1	—	—
4	I4	I3	I2	I1	—
5	I5	I4	I3†	I2	I1
6	I6	I5	I4	I3	I2
7	I7	I6	I5	I4	I3
8	I8	I7	I6	I5	I4

†*At cycle 5, I3 (`add x1, x2, x3`) is in EX. I1 (`addi x2`) is in WB stage, so x2=5 is forwarded from WB. I2 (`addi x3`) is in MEM stage, so x3=10 is forwarded from MEM. The ALU correctly computes 5 + 10 = 15.*

This is the essence of pipelining in action—five instructions are being processed simultaneously, and the forwarding unit ensures that data dependencies are resolved without stalling (except for the unavoidable load-use case).

15 Branch Execution in Detail

Let us trace the branch instruction (I13: `beq x4, x5, +8`) through the pipeline:

1. **Cycle 13**: I13 is fetched from instruction memory at PC = `0x30`. The branch predictor checks BHT[12] (since `0x30[5:2]` = 12). The counter is at `00` (Strongly Not Taken), so the predictor predicts **not taken**. PC advances to `0x34` as normal.
2. **Cycle 14**: I13 enters the ID stage. The branch comparator (with forwarding) compares x4=10 and x5=10, finds them equal, and sets `branch_taken = 1`. Since the predictor said not-taken but the branch is actually taken, a **misprediction** is detected. Meanwhile, I14 (`beq x0, x0, 0`) has already been fetched speculatively.
3. The branch target is computed: `0x30` + $(4 \ll 1)$ = `0x30` + 8 = `0x38`.
4. The IF/ID register is flushed, killing I14.
5. The BHT entry at index 12 is updated: `00` → `01` (Weakly Not Taken).
6. **Cycle 15**: The PC is now `0x38`, and I15 (`add x13, x1, x10`) is fetched. I14 is gone—it never executes.

The 1-cycle branch penalty means we wasted one fetch cycle on the misprediction. The branch predictor would do better on subsequent encounters with the same branch: after one more taken execution, the counter would reach `10` (Weakly Taken) and start predicting correctly.

16 Module Hierarchy and File Organization

Table 10: Complete Module List

File	Module(s)	Purpose
add.v	adder, adder64	1-bit full adder and 64-bit ripple-carry adder
sub.v	sub64	64-bit subtractor (2's complement)
and.v	and_gate	64-bit bitwise AND
or.v	or_gate	64-bit bitwise OR
xor.v	xor_gate	64-bit bitwise XOR
zero64.v	zero_check	64-bit zero detection
alu.v	alu_64_bit	Main 64-bit ALU
alu_control.v	alu_control	ALU opcode decoder
pc.v	pc	Program counter register
pc_add.v	pc_add	PC + 4 adder
pc_mux.v	pc_mux	Next-PC multiplexer
instruction_mem.v	instruction_mem	Instruction ROM
register_file.v	register_file	32×64-bit register file
control_unit.v	control_unit	Main control unit
imm_gen.v	imm_gen	Immediate generator
data_mem.v	data_mem	Data memory (1 KB)
mux2.v	mux2	64-bit 2-to-1 multiplexer
shift_left1.v	shift_left1	Left shift by 1
if_id.v	if_id	IF/ID pipeline register
id_ex.v	id_ex	ID/EX pipeline register
ex_mem.v	ex_mem	EX/MEM pipeline register
mem_wb.v	mem_wb	MEM/WB pipeline register
forwarding_unit.v	forwarding_unit	Data forwarding logic
hazard_detection_unit.v	hazard_detection_unit	Load-use hazard detector
branch_predictor.v	branch_predictor	2-bit saturating counter BHT (16 entries)
seq_processor.v	seq_processor	Top-level processor (wrapper)
pipe_tb.v	pipe_tb	Pipeline testbench

17 What Changed: Sequential to Pipelined

Here is a summary of everything we added or changed when we converted the sequential processor to the pipelined version:

Table 11: Sequential vs. Pipelined Design Comparison

Aspect	**Sequential**	**Pipelined**
Instruction execution	1 instruction per cycle (all stages in 1 cycle)	1 instruction *completed* per cycle (5 stages overlapped)
Pipeline registers	None	4 registers: IF/ID, ID/EX, EX/MEM, MEM/WB
Forwarding	Not needed	Forwarding unit with 3-input muxes on ALU inputs
Hazard detection	Not needed	Load-use hazard detector generates stalls
Branch resolution	No need as instruction is fetched after completing the other	In ID stage with branch comparator forwarding (1-cycle penalty on misprediction)
Branch prediction	Not applicable	2-bit saturating counter predictor with 16-entry BHT
Throughput	Limited by critical path	Up to 5× improvement (ideal)
Clock frequency	Low (long critical path)	Higher (shorter stage delays)
Hardware cost	Minimal	4 pipeline registers + forwarding + hazard detection

17.1 Key Design Decisions

1. **Branch in ID stage**: We moved the branch comparison from the EX stage (standard textbook approach) to the ID stage. This reduced the branch penalty from 2 cycles to 1 cycle but required adding an equality comparator in the ID stage.

2. **Forwarding priority**: MEM-stage forwarding takes priority over WB-stage forwarding. If both stages produce a value needed by EX, the MEM value is more recent and therefore correct.

3. **Register file internal forwarding**: The register file itself handles the case where WB writes and ID reads the same register in the same cycle, forwarding the write data directly to the read port.

4. **Stall via ID/EX flush**: Instead of adding a separate "bubble" multiplexer, we reuse the ID/EX register's flush input. When a stall is needed, the ID/EX register is flushed to all zeros (a NOP), effectively inserting a bubble into the pipeline.

5. **Immediate halt**: When the IF stage fetches an all-zeros instruction (`0x00000000`), the processor halts on the next clock edge. It does not automatically drain the pipeline — the programmer must place explicit NOP instructions before the halt sentinel to ensure all real instructions complete.

6. **2-bit branch predictor**: We chose a 2-bit saturating counter scheme over a simpler 1-bit predictor because it tolerates one "anomalous" outcome without changing prediction direction. The 16-entry BHT indexed by PC[5:2] keeps the hardware cost minimal while covering common loop and branch patterns.

7. **Branch comparator forwarding**: To ensure the ID-stage branch comparator sees the latest register values, we added forwarding paths from the EX stage (ALU result, excluding loads) and MEM stage (ALU result or loaded data). Without this, branches depending on a value computed just one or two instructions earlier would use stale data.

18 Simulation and Results

The processor was simulated using the testbench `pipe_tb.v`, which instantiates the top-level `seq_processor` module and monitors key signals every clock cycle.

18.1 Simulation Setup

- **Clock**: 10 ns period (5 ns high, 5 ns low)
- **Reset**: Held high for 20 ns (2 cycles), then released
- **Monitoring**: PC, instruction, register reads, ALU result, and write-back signals displayed every cycle
- **Termination**: On halt detection or after 200 cycles (timeout)

18.2 Results Summary

The simulation produced the correct output for all 15 instructions:

- All R-type operations (ADD, SUB, AND, OR) computed the correct results.
- Negative results from SUB ($5 - 10 = -5$) were handled correctly in two's complement.
- Both store/load pairs correctly wrote to and read from data memory using base+offset addressing.
- The BEQ instruction correctly took the branch ($x4 = x5 = 10$), flushing the speculatively fetched I14.
- The branch predictor initially predicted not-taken (counter at `00`), causing a 1-cycle misprediction penalty that was correctly recovered.
- Register x13 correctly held 30 ($= 15 + 15$), the final result computed after the branch.
- The forwarding unit correctly handled all data dependencies without any unnecessary stalls.

- The instruction counter finished at 20, matching the expected count (15 real + 4 NOPs + 1 flush slot).
- The final register file dump (`register_file.txt`) matched the expected values exactly.

19 Challenges and Lessons Learned

Building a pipelined processor is not just about adding pipeline registers. Here are some of the real challenges we faced:

1. **Forwarding edge cases**: Getting the forwarding logic right for consecutive dependent instructions was tricky. We had to carefully handle the priority between MEM and WB forwarding, and make sure we never forwarded from x0.
2. **Stall + flush interaction**: When a load-use hazard triggers a stall, the IF/ID register must hold and the ID/EX register must flush. Getting these two actions to happen simultaneously without corrupting the pipeline state took careful testing.
3. **Branch comparator forwarding**: Moving branch resolution to the ID stage introduced a subtle issue: the register values used for comparison must already be correct. If the branch depends on a value computed just one or two instructions earlier, that value has not yet reached the register file. We solved this by adding dedicated forwarding paths from the EX and MEM stages to the branch comparator in ID.
4. **Branch predictor integration**: The predictor must make its prediction in the IF stage (before the instruction is decoded) and update in the ID stage (after resolution). Coordinating the prediction carry-through via the IF/ID register, suppressing BHT updates during stalls, and masking flushes during stalls required careful signal gating.
5. **Register file write-read ordering**: Since the register file writes on the positive clock edge and reads combinationally, we needed internal forwarding to handle the case where WB writes and ID reads the same register in the same cycle.

20 Conclusion

We successfully designed, implemented, and verified a 5-stage pipelined 64-bit RISC-V processor with a 2-bit branch predictor in Verilog. Starting from a working sequential design gave us a solid foundation and confidence in our individual components. The pipeline transformation added four pipeline registers, a forwarding unit with MEM and WB forwarding paths, a hazard detection unit for load-use stalls, branch resolution in the ID stage with branch comparator forwarding, and a 16-entry BHT-based branch predictor.

The processor correctly executes a 15-instruction test program covering ADD, SUB, AND, OR, ADDI, LD, SD, and BEQ. Data forwarding resolves dependencies without stalling, load-use hazards incur only a single-cycle bubble, and branch mispredictions are

corrected in one cycle. The final register file contents and instruction count match the expected values, confirming the correctness of our implementation.

This project gave us hands-on experience with the real complexities of processor design—not just the "textbook" pipeline diagram, but the nitty-gritty details of hazard detection, forwarding priority, branch prediction, misprediction recovery, and pipeline control. These are the problems that make computer architecture both challenging and deeply rewarding.